UNIT III

Constructors, Destructors and Operator Overloading

Solved Debugging Questions, Corrected C++ Programs, Pseudo-Code and Dry Runs

SECTION A – DEBUGGING QUESTIONS

Each answer identifies the error, explains the C++ rule, provides corrected code, and shows representative output. Memory addresses shown by programs vary between executions.

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

Bug location: Line 17 – DynamicString(DynamicString source).

Classification: Invalid copy-constructor signature / recursive copy construction.

Root cause: A copy constructor must receive the source object by reference. If its parameter is passed by value, creating the parameter itself requires copying the argument. That copy again needs the same copy constructor, creating an impossible recursive requirement. The compiler therefore rejects the declaration; this is not a normal runtime recursion.

Correction: Use DynamicString(const DynamicString& source). The & avoids making another copy, while const allows copying from const objects and temporary/read-only sources without modifying the original.

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char* buffer;
    int length;

public:
    DynamicString(const char* str) {
        length = static_cast<int>(std::strlen(str));
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    DynamicString(const DynamicString& source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << '\n';
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
    return 0;
}
```

Representative output:

```
Antigravity C++
```

Note: This implementation performs a deep copy: s1 and s2 own separate character arrays.

Question 2: Shallow Copy and Double-Free Runtime Crash

Bug location: The class has no custom copy constructor; processAudio(AudioBuffer buf) passes the object by value.

Classification: Shallow-copy pointer aliasing, dangling pointer, and double deletion.

Root cause: The compiler-generated copy constructor copies each data member. For a raw pointer, it copies only the address, not the allocated array. Thus track.samples and buf.samples point to the same heap block. When buf is destroyed at the end of processAudio, it deletes the block. track.samples then dangles, and track's destructor later tries to delete the same block again.

Correction: Add a deep-copy constructor. The example below also uses a const-reference parameter for processAudio because the function only reads the buffer.

```
#include <iostream>

class AudioBuffer {
public:
    int* samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i)
            samples[i] = i * 10;
        std::cout << "Allocated: " << samples << '\n';
    }

    AudioBuffer(const AudioBuffer& other) : capacity(other.capacity) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i)
            samples[i] = other.samples[i];
        std::cout << "Deep copy allocated: " << samples << '\n';
    }

    ~AudioBuffer() {
        std::cout << "Deallocating: " << samples << '\n';
        delete[] samples;
    }
};

void processAudio(const AudioBuffer& buf) {
    std::cout << "Processing sample 0: " << buf.samples[0] << '\n';
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << '\n';
    return 0;
}
```

Representative output (addresses are illustrative):

```
Allocated: 0x1000
Processing sample 0: 0
Back in main, sample 0: 0
Deallocating: 0x1000
```

The deep-copy constructor is included to make copying safe if an AudioBuffer is copied by value. With the const-reference function parameter shown here, no copy is made during this particular call.

Rule of Three: when a class manually manages a resource and defines a destructor, it commonly also needs a copy constructor and copy-assignment operator. Modern C++ often uses RAII containers such as std::vector instead.

Question 3: Stream Insertion Operator as a Member Function

Bug location: Line 11 declares operator<< as a member of Fraction.

Classification: Incorrect operand placement for stream insertion.

Root cause: A binary member operator expression lhs << rhs is translated as lhs.operator<<(rhs). Therefore std::cout << f1 tries to find operator<< as a member of std::ostream, with Fraction as its argument. Fraction's member operator is instead called as f1 << std::cout, which reverses the conventional operands.

Correction: Define operator<< as a non-member friend. Then the left operand can be std::ostream and the right operand can be Fraction.

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    friend std::ostream& operator<<(std::ostream& os,
                                    const Fraction& value) {
        os << value.numerator << "/" << value.denominator;
        return os;
    }
};

int main() {
    Fraction f1(3, 4);
    std::cout << f1 << std::endl;
    return 0;
}
```

Output:

```
3/4
```

Returning std::ostream& allows stream chaining, such as std::cout << f1 << std::endl.

Question 4: Prefix vs. Postfix Increment Signature Inversion

Bug location: The two operator++ declarations have their intended meanings reversed.

Classification: Incorrect overload signatures and return semantics.

- Prefix: StepCounter& operator++() – no dummy parameter; increment first and return the modified object by reference.
- Postfix: StepCounter operator++(int) – dummy int distinguishes postfix; save old value, increment the object, and return the old snapshot by value.

The dummy int is a language convention used only to distinguish postfix from prefix; callers do not supply it.

```
#include <iostream>
```

```
class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    StepCounter& operator++() {       // Prefix: ++obj
        ++steps;
        return *this;
    }

    StepCounter operator++(int) {     // Postfix: obj++
        StepCounter old(*this);
        ++steps;
        return old;
    }

    void display() const {
        std::cout << "Steps: " << steps << '\n';
    }
};

int main() {
    StepCounter c1(10);
    StepCounter c2 = ++c1;
    c2.display();
    c1.display();

    StepCounter c3 = c1++;
    c3.display();
    c1.display();
    return 0;
}
```

Output:

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

Prefix returns the updated object and supports chaining. Postfix returns the saved old value, so it normally returns by value.

Question 5: Violation of Fundamental Operator Overloading Rules

Bug locations: Line 8 attempts to overload operator.; Line 14 overloads + for two built-in int operands; Line 19 invents operator**.

Classification: Prohibited operator overload, built-in-only operands, and invalid operator token.

Root cause and rules:

- The member-access dot operator (.), scope-resolution operator (::), conditional operator (?:), sizeof, and pointer-to-member operator (.*) cannot be overloaded. The standard also prohibits overloading several other operators such as typeid, alignof, and the named cast operators.
- An overloaded operator must have at least one operand of a user-defined class or enumeration type. int + int cannot be redefined.

- Programmers cannot invent new operator tokens such as **. Overloading does not change an operator's precedence, associativity, or number of operands.

Corrected legal design: overload == and + for MathVector, and implement exponentiation as an ordinary member function.

```
#include <iostream>
#include <cmath>

class MathVector {
public:
    double x, y;

    MathVector(double a = 0.0, double b = 0.0) : x(a), y(b) {}

    bool operator==(const MathVector& other) const {
        return x == other.x && y == other.y;
    }

    MathVector operator+(const MathVector& other) const {
        return MathVector(x + other.x, y + other.y);
    }

    MathVector power(int exponent) const {
        return MathVector(std::pow(x, exponent),
                          std::pow(y, exponent));
    }

    void display() const {
        std::cout << "(" << x << ", " << y << ")";
    }
};

int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);
    MathVector sum = v1 + v2;
    MathVector squared = v1.power(2);

    std::cout << "Equal: " << (v1 == v2) << '\n';
    std::cout << "Sum: ";
    sum.display();
    std::cout << "\nPower 2: ";
    squared.display();
    std::cout << '\n';
    return 0;
}
```

Output:

```
Equal: 1
Sum: (4, 6)
Power 2: (4, 9)
```

SECTION B – PSEUDO-CODE WRITING QUESTIONS

The following structured pseudo-code follows the question bank's required lifecycle and operator notation. Dry-run values are calculated from the given sample inputs.

Question 1: Dynamic String Engine with Deep Copy and Overloaded +

Structured pseudo-code

```
CLASS DynStr
    PRIVATE:
        dataPtr : POINTER TO CHARACTER
        charCount : INTEGER

    CONSTRUCTOR(sourceString AS STRING)
        charCount <- LENGTH(sourceString)
        dataPtr <- ALLOCATE ARRAY[charCount + 1]
        COPY sourceString INTO dataPtr
        dataPtr[charCount] <- NULL

    COPY CONSTRUCTOR(REF other AS DynStr)
        charCount <- other.charCount
        dataPtr <- ALLOCATE ARRAY[charCount + 1]
        FOR i <- 0 TO charCount - 1
            dataPtr[i] <- other.dataPtr[i]
        END FOR
        dataPtr[charCount] <- NULL

    OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
        total <- charCount + rhs.charCount
        temp <- ALLOCATE ARRAY[total + 1]
        COPY this.dataPtr INTO temp
        APPEND rhs.dataPtr TO temp
        result <- DynStr(temp)
        DEALLOCATE temp
        RETURN result

    DESTRUCTOR()
        DEALLOCATE dataPtr
        dataPtr <- NULL

    DISPLAY()
        PRINT dataPtr
END CLASS

MAIN
    strA <- DynStr("Hello ")
    strB <- DynStr("World!")
    strC <- strA + strB
    strC.DISPLAY()
END MAIN
```

Dry run: strA = "Hello " (6 characters) and strB = "World!" (6 characters).

Step	Operation	Heap / content
1	Construct strA	Allocate 7 characters; stores Hello plus null terminator
2	Construct strB	Allocate 7 characters; stores World! plus null terminator
3	Add lengths	6 + 6 = 12 characters
4	Concatenate	Temporary buffer size = 13; content

		Hello World!
5	Construct result	strC owns its own 13-character buffer (deep copy)
6	Cleanup	Temporary buffer released; each object's own buffer released at destruction

Output:

```
Hello World!
```

Memory note: The returned object must own independent storage. A practical C++ implementation should also define copy assignment or use std::string to avoid Rule-of-Three/Rule-of-Five problems.

Question 2: Complex Number System with Constructor Overloading and Operators

Structured pseudo-code

```
CLASS Complex
    PRIVATE:
        real : REAL
        imag : REAL

    CONSTRUCTOR()
        real <- 0.0
        imag <- 0.0

    CONSTRUCTOR(r AS REAL)
        real <- r
        imag <- 0.0

    CONSTRUCTOR(r AS REAL, i AS REAL)
        real <- r
        imag <- i

    OPERATOR +(REF other AS Complex) RETURNS Complex
        RETURN Complex(real + other.real, imag + other.imag)

    OPERATOR *(REF other AS Complex) RETURNS Complex
        newReal <- real * other.real - imag * other.imag
        newImag <- real * other.imag + imag * other.real
        RETURN Complex(newReal, newImag)

    OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
        RETURN (real = other.real AND imag = other.imag)

    DISPLAY()
        PRINT real, " + ", imag, "i"
END CLASS
```

Dry run: C1 = 3 + 2i and C2 = 1 + 4i.

Calculation	Working	Result
Real part	(3 × 1) − (2 × 4)	3 − 8 = −5
Imaginary part	(3 × 4) + (2 × 1)	12 + 2 = 14
Product	Real + imaginary part	−5 + 14i

Final answer: C1 × C2 = −5 + 14i.

Optional addition check: (3 + 2i) + (1 + 4i) = 4 + 6i.

Question 3: Financial Currency Class with Stream I/O and Unary Negation

Structured pseudo-code

```
CLASS Currency
    PRIVATE:
        dollars : INTEGER
        cents : INTEGER

    CONSTRUCTOR(d = 0, c = 0)
        totalCents <- d * 100 + c
        dollars <- totalCents DIV 100
        cents <- totalCents MOD 100
        // Keep a consistent signed total-cents representation.

    OPERATOR -() RETURNS Currency
        totalCents <- dollars * 100 + cents
        RETURN CurrencyFromTotalCents(-totalCents)

    FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)
        PRINT "$", cur.dollars, ".", TWO_DIGIT(cur.cents)
        RETURN outStream

    FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)
        READ d, c
        cur <- Currency(d, c)
        RETURN inStream

    DISPLAY()
        PRINT formatted currency
END CLASS

MAIN
    cur <- Currency(5, 275)
    PRINT cur
    debt <- -cur
    PRINT debt
END MAIN
```

Dry run for d = 5 and c = 275:

Step	Calculation	Value
1	Convert to total cents	5 × 100 + 275 = 775 cents
2	Normalize dollars	775 DIV 100 = 7 dollars
3	Normalize cents	775 MOD 100 = 75 cents
4	Display original	$7.75
5	Apply unary minus	−775 cents
6	Display negated value	−$7.75

Important implementation note: The question says negative cents must be handled. The safest representation is one signed total-cents integer internally, then format its absolute dollar and cent parts with a separate sign. The pseudo-code above describes that normalization approach.

Question 4: Polynomial Expression with Subscript [] and Destructor

Structured pseudo-code

```
CLASS Polynomial
```

```
    PRIVATE:
        degree : INTEGER
        coeffArray : POINTER TO REAL

    CONSTRUCTOR(deg AS INTEGER)
        degree <- deg
        coeffArray <- ALLOCATE ARRAY[degree + 1]
        FOR i <- 0 TO degree
            coeffArray[i] <- 0.0
        END FOR

    COPY CONSTRUCTOR(REF other AS Polynomial)
        degree <- other.degree
        coeffArray <- ALLOCATE ARRAY[degree + 1]
        FOR i <- 0 TO degree
            coeffArray[i] <- other.coeffArray[i]
        END FOR

    OPERATOR [](index AS INTEGER) RETURNS REF REAL
        IF index < 0 OR index > degree
            REPORT "Index out of range"
        END IF
        RETURN coeffArray[index]

    OPERATOR +(REF other AS Polynomial) RETURNS Polynomial
        maxDegree <- MAX(degree, other.degree)
        result <- Polynomial(maxDegree)
        FOR i <- 0 TO maxDegree
            leftValue <- 0.0
            rightValue <- 0.0
            IF i <= degree
                leftValue <- coeffArray[i]
            END IF
            IF i <= other.degree
                rightValue <- other.coeffArray[i]
            END IF
            result[i] <- leftValue + rightValue
        END FOR
        RETURN result

    DESTRUCTOR()
        DEALLOCATE coeffArray
        coeffArray <- NULL
END CLASS
```

Coefficient convention: coeffArray[i] stores the coefficient of x^i.

Dry run: P1(x) = $3x^2 + 2x + 5$ and P2(x) = $4x^3 + x^2 + 7$.

Power	P1 coefficient	P2 coefficient	Sum
x^0	5	7	12
x^1	2	0	2
x^2	3	1	4
x^3	0	4	4

Final polynomial: P1(x) + P2(x) = $4x^3 + 4x^2 + 2x + 12$.

Bounds-check note: In real C++, operator[] normally returns a reference. It should not silently return an invalid reference; use a checked accessor or throw std::out_of_range when the index is invalid.

Question 5: 2D Vector with Prefix/Postfix Increment and Compound Assignment

Structured pseudo-code

```
CLASS Vector2D
    PRIVATE:
        x : REAL
        y : REAL

    CONSTRUCTOR(xValue = 0.0, yValue = 0.0)
        x <- xValue
        y <- yValue

    OPERATOR ++() RETURNS REF Vector2D
        x <- x + 1.0
        y <- y + 1.0
        RETURN THIS

    OPERATOR ++(dummy INTEGER) RETURNS Vector2D
        oldValue <- COPY OF THIS
        x <- x + 1.0
        y <- y + 1.0
        RETURN oldValue

    OPERATOR +=(REF rhs AS Vector2D) RETURNS REF Vector2D
        x <- x + rhs.x
        y <- y + rhs.y
        RETURN THIS

    DISPLAY()
        PRINT "(" , x , ", " , y , ")"
END CLASS

MAIN
    v1 <- Vector2D(5.0, 8.0)
    v2 <- ++v1
    v3 <- v1++
    DISPLAY v1, v2, v3
END MAIN
```

Dry run:

Statement	v1	v2	v3
Initial state	(5.0, 8.0)	Not assigned	Not assigned
v2 <- ++v1	(6.0, 9.0)	(6.0, 9.0)	Not assigned
v3 <- v1++	(7.0, 10.0)	(6.0, 9.0)	(6.0, 9.0)

Explanation: Prefix increments v1 first, then v2 receives the updated value. Postfix saves the old value (6.0, 9.0), increments v1 to (7.0, 10.0), and returns the saved snapshot to v3.

Final values: v1 = (7.0, 10.0), v2 = (6.0, 9.0), v3 = (6.0, 9.0).

QUICK REVISION – IMPORTANT C++ RULES

Concept	Correct rule / signature
Copy constructor	ClassName(const ClassName& other)
Deep copy	Allocate a new buffer and copy each element
Destructor	~ClassName(); release owned resources
Stream insertion	friend std::ostream& operator<<(std::ostream&, const ClassName&)
Stream extraction	friend std::istream& operator>>(std::istream&, ClassName&)
Prefix increment	ClassName& operator++()
Postfix increment	ClassName operator++(int)
Unary minus	ClassName operator-() const
Subscript	Type& operator[](std::size_t index)
Compound assignment	ClassName& operator+=(const ClassName& rhs)
Operator constraints	At least one operand must be user-defined; cannot invent tokens or alter precedence/arity

Exam tip: For every dynamic-memory class, clearly show who owns the allocated memory, when a fresh allocation is made during copying, and exactly where delete[] is called. For overloaded operators, write the signature first and then explain which operand is the left-hand side.